

GPU Native Scan — Full Report

Branch `feature/gpu-native-scan-task` on `sirius-dev`. Status as of 2026-04-16.
Audience: anyone picking up or reviewing this work.

TLDR

We replaced DuckDB's CPU `duckdb_scan_task` with a Sirius-native scan that decodes `.duckdb` block storage directly on the GPU. The branch adds four major pieces (scan task, GPU decode kernels, `Pin()` bypass, parallel scheduling) and a long tail of smaller optimizations.

Benchmark	Before	After	Change
TPC-H SF=10 warm (RTX6000)	10.49s	5.37s	−49%
TPC-H SF=100 warm (GH200)	6.26s	5.33s	−15%, now 1.33× CPU
ClickBench 10M warm (RTX6000)	3.37s	2.10s	−38%
ClickBench 100-shard warm (GH200)	OOM / 29.3s	5.00s	−83%
TPC-H SF=100 cold (GH200)	16.4s/q	2.4s/q	6.9×

Where the remaining cost lives (Q1 SF=100 warm): GPU kernel time 608ms, of which our decode kernels are **11%** (the rest is cuDF `hash_group_by` / CUB internals). CPU-side API + sync overhead is 3× larger than actual GPU compute. We are no longer bottlenecked on scan.

Piece 1 — `gpu_native_scan_task`

High-level

DuckDB's normal scan flow:

```
duckdb_scan_task (CPU)
  → Pin() block
  → CPU decompress into DataChunk
  → hand DataChunk to next pipeline operator
Sirius: H2D copy the DataChunk → GPU
```

What we replaced it with:

```
gpu_native_scan_task (Sirius)
  → walk segment tree directly
  → get mmap'd block pointer (no Pin)
  → H2D the compressed block
  → GPU decode kernel produces cudf columns
  → hand cudf table directly to next pipeline operator
```

The CPU decompression step is gone — blocks arrive on the GPU in their on-disk layout and decode there. This matters because at SF=100 a single Q1 column is ~29K segments and CPU decompression was the wall bottleneck.

Entry point: `src/op/scan/gpu_native_scan_task.cpp`. Viability check rejects tables that include compression types we haven't implemented a GPU decoder for, and falls back to the legacy `duckdb_scan_task`. Today everything TPC-H needs and all of ClickBench's supported queries fit.

Impactful optimizations

- **Row-group batching by `scan_task_batch_size` (500Mi default).** Initial version accumulated all segments of a table into one task, which OOM'd on SF=100 and had poor pipeline parallelism. We batch by byte size so each task has a bounded working set. 500Mi was tuned empirically: 100Mi is 19% slower at SF=10, 2Gi is equivalent to 500Mi but riskier under pressure.
- **Bulk pre-transfer H2D (commit 189b707).** Instead of one `cudaMemcpyAsync` per segment, we sort segments by `block_id` and issue multi-MB contiguous transfers (the mmap layout is naturally contiguous across `block_id`). Reduced ~53K small H2D calls to a few dozen big ones, lifting per-transfer size above the 4MB PCIe efficiency cliff.
- **Skip-redundant-H2D (commits 9111177 / a6ea8e7).** Constant, blockless, and same-block-twice segments don't need their own transfer. We dedupe at task build time.
- **Bounded self-continuation (`MAX_BATCHES_PER_TASK=4` , in 5e990a8).** When one scan task finishes it can schedule a replacement task for the next batch, which restores pipeline interleaving that parallel scan tasks broke. The bound prevents one task from sucking all batches and OOM'ing on ClickBench 100-shard warm.

Miscellaneous

- Per-column decode-temp buffer pre-allocation (`ce984ee`) so decode kernels don't pay allocator overhead per segment.
- Row-group pruning + `projection_ids fix` (`ea8d1b7`) so we only walk segments of requested columns.
- Correct batch-size computation from actual segment metadata, not type-width estimates (`bde53e1`). Type estimates were 2× off on dict columns — we'd either leave capacity on the table or OOM.

Piece 2 — GPU Decode Kernels

High-level

We support six compression types natively on the GPU. Each kernel takes a raw `.duckdb` block pointer + segment metadata and writes decompressed output into a cudf column buffer.

Type	Fixed-width	String	File
Uncompressed	✓	✓	direct H2D memcpy
Constant	✓ (blockless + block-backed)	✓	extracted from stats
BitPacking	✓ (int8/16/32/64)	—	<code>gpu_decode_bitpacking.cu</code>
Dictionary	—	✓	<code>gpu_decode_dictionary.cu</code>
FSST	—	✓	<code>gpu_decode_fsst.cu</code>
RLE	✓ (int8/16/32/64)	—	<code>gpu_decode_rle.cu</code>

Orchestration is in `gpu_native_decode.cu` (`decode_string_column` + `decode_fixed_column`). The caller hands over a list of segments per column; the orchestrator picks the right decoder per segment.

Impactful optimizations

- **Fused GPU-side bitpacking metadata parse (0c3aeca).** Before: CPU parsed DuckDB's per-group bitpacking metadata (width, min_delta, frame-of-reference) and passed it as separate H2D arrays. After: the block is on GPU, the kernel parses metadata itself. Saves ~1 CPU-side prep pass per segment. **–26% TPC-H SF=10 warm** (10.49s → 7.77s).
- **Batched bitpacking decode (6de87fb).** 48K kernel launches/query collapsed to ~1K by treating the 2048-row metadata group as the CTA unit. **–13% TPC-H SF=100 suite.**
- **Batched two-pass string decode (6bda3b8).** String decode is inherently 2 passes: compute output lengths → allocate → gather. Naive implementation does per-segment sync + kernel launch pair (2 syncs × 17 segments = 34 syncs per column). Batched version issues pass-1 across all segments, one global `ExclusiveSum` for offsets, pass-2 across all segments. **Eliminates per-segment sync and launch overhead.**
- **Kernel opts bundle (f778206).** Dict string lookup now fuses length and gather pointer calculation in one pass. RLE decode uses shared memory for the prefix sum. Null count moved to GPU (was a `cudaMemcpy` per segment from CPU). **–18% ClickBench warm.**

Miscellaneous

- Blockless CONSTANT fix (a270be0) — constant segments have no block, just stats. Extract value from `NumericStats::GetMin` .
- `bitpacking` int8/int16 handlers (a270be0) — were missing, falling through to slow path.
- Per-column batched null count (b7ce106) — one `cudaMemcpyAsync` per table, not per segment. Architecturally correct even if wall-clock neutral at SF=10.

Piece 3 — Direct `mmap` Bypass of `Pin()`

High-level

`BufferManager::Pin()` is DuckDB's per-block-load gate. For a read-only `.duckdb` file, the block data is already visible in the `mmap`'d file — `Pin()` adds mutex + refcount + `FileBuffer` copy that we don't need.

Measured overhead at SF=100 Q1: **470ms** of a 630ms wall was `Pin()` .

Our scan path `mmap`'s the `.duckdb` file once, discovers the block layout (`BLOCK_START + block_id * alloc_size + header`), validates it against `Pin()` 'd output once via `std::call_once` , and reads blocks directly from `mmap` thereafter.

File: `src/op/scan/direct_block_scan.cpp` .

Impactful optimizations

- **Atomic fast path (this session).** Prior version held a `std::mutex` on every block lookup to find the per-database `mmap` state. With parallel scan tasks, this mutex was the new bottleneck — **-8% warm SF=10** just from switching to `std::atomic<mmap_file_state*>` + DCL. Benchmarked equivalent to DuckDB's upstream `GetDirectBlockPointer()` within 2-3% noise.
- **Removed `MADV_POPULATE_READ` (938898f)**. Earlier version prefaulted the entire database file on first `mmap` to work around what we thought was a CUDA staging-thread segfault on lazy faults. After investigation, the segfault was actually caused by broken block offset math (fixed in 5e990a8), not lazy faults. Prefaulting was polluting the OS page cache with the full DB file — **6.9x faster cold TPC-H SF=100 on GH200** after removal (16.4s/q → 2.4s/q avg). Lazy `mmap` works on both PCIe (CUDA's pinned staging handles faults as normal CPU memcpy) and GH200 (ATS transparently).
- **Thread-safe init via `std::call_once`** . Layout discovery (`alloc_size` , `hdr_size` , `block_start=4096*3`) runs once per process against a real `Pin()` 'd segment; value is then stored in a globally readable struct. The

original implementation had a data race on these globals that caused Q2 segfaults under parallel scan.

Miscellaneous

- `SIRIUS_DISABLE_MMAP=1` env flag forces the `Pin()` path for A/B benchmarking.
- Runtime validation on first segment: `memcmp(mmap_ptr, pin_ptr, 64)`. If it fails we log loud and disable mmap for this session — no silent wrong answers.
- Per-BlockManager cache keyed by pointer (not path) so multi-DB attach works and avoids a `GetDBPath()` string copy on the hot path.

Piece 4 — Parallel + Batched Scheduling

High-level

DuckDB runs scan tasks on its pipeline threads. Sirius has a dedicated `duckdb_scan` thread pool (2 threads by default). We launch multiple concurrent `gpu_native_scan_task` instances against the same query so the pool stays full, and each task self-continues through its row-group batch.

Impactful optimizations

- **Parallel scan tasks (685682c).** Launch N scan tasks up front (N = scan pool size), each claims its own row-group range. Before this the pipeline only ever had one scan task in flight at a time, serializing everything behind pin + H2D.
- **2 threads is the sweet spot.** Measured at SF=10:

threads	batch	warm
2	500Mi	5.58s ← best
2	2Gi	5.61s
4	100Mi	8.42s (GPU contention)
8	500Mi	6.06s

More threads hurt because all scan tasks target the same GPU and contend for kernel launch / RMM allocation. The `num_threads: 2` default in `sirius.yaml` matters.

- **500Mi batch size default.** 100Mi is 19% slower, 2Gi is riskier under memory pressure. Set in `src/include/sirius_config.hpp:32`.

Miscellaneous

- Self-continuation replaces task on completion rather than blocking for next — keeps the pipeline fed.
- `MAX_BATCHES_PER_TASK=4` cap prevents a single parallel task from hoarding all row groups; ClickBench 100-shard warm was OOM'ing without this bound.

What we tested and rejected

This is important context — these aren't missing features, they were evaluated and measured worse:

Approach	Result	Why
H2D/compute dual-stream interleaving	−8% regression SF=10	Overhead exceeds overlap gain at current batch size. Reverted e3e2719 .
<code>cudaMemcpyBatchAsync</code>	+52% slower	Changes internal copy semantics — serializes what was pipelined
Pinned memory H2D staging	Slower	CPU memcpy into write-combining memory is 8 GB/s, PCIe H2D from pageable is 12 GB/s
GH200 zero-copy scan	+15% slower Q1	Kernels read from host at 615 GB/s vs HBM 3,400 GB/s
4+ scan threads	Slower	GPU contention from concurrent scan tasks
<code>MADV_POPULATE_READ</code> prefault	Polluted page cache	Lazy faults work, prefault helped nothing
In-place string compaction (3 variants)	Correctness failures	Overlapping src/dest

Do not re-litigate these without fresh data.

Where to pick this up

Ordered by where we'd spend next-sprint engineering:

1. **Execution-side operator optimization** (join / agg / sort / expr). At SF=100 warm Q1, our decode is 11% of GPU kernel time and cuDF `hash_group_by_single_pass_shmem_aggregate` is 56%. Scan is close to its limit; the next 2x lives downstream.

2. **String decode GPU utilization** — dict/FSST gather launches with grid = 5-17 CTAs (SM util 13-22%). One CTA per 256-row chunk within a segment would bring it to 500+ CTAs. Expected: **~219ms saved on Q1 SF=100**. Sketch in `docs/scan-optimization-guide.md` §0.
3. **Vectorized string writes** — dict/FSST gather currently byte-copies; `memcpy` compiles to vector loads/stores. Expected 1.5-2× on gather kernels. `docs/scan-optimization-guide.md` §1.
4. **DICT_FSST decode-once** — for DICT_FSST columns, FSST-decode only the dictionary (small), then gather from decoded table. ClickBench Q21 est. ~2300ms → ~300ms.
5. **Ring buffer + pinned + dual-stream overlap** — ~58ms on Q1 SF=10 (−23% total). Design doc in `active_scan_interleaving.md`. Worth doing but lower leverage than items 1-4.
6. **GPU table cache (`table_gpu`)** — only matters if we expect repeated queries against the same table in a session. Warm pin is already 1.5ms. Defer.

Key files

File	Role
<code>src/op/scan/gpu_native_scan_task.cpp</code>	Scan task, viability, batch sizing
<code>src/op/scan/direct_block_scan.cpp</code>	mmap bypass + atomic fast path
<code>src/cuda/scan/gpu_native_decode.cu</code>	Column-level decode orchestrator
<code>src/cuda/scan/gpu_decode_bitpacking.cu</code>	BitPacking (int8/16/32/64, fused)
<code>src/cuda/scan/gpu_decode_dictionary.cu</code>	Dictionary strings
<code>src/cuda/scan/gpu_decode_fsst.cu</code>	FSST strings (6-kernel pipeline)
<code>src/cuda/scan/gpu_decode_rle.cu</code>	RLE (prefix sum + binary-search expand)
<code>src/include/cuda/scan/gpu_decode.cuh</code>	Decode API + <code>string_decode_temp</code>
<code>src/include/sirius_config.hpp:32</code>	<code>DEFAULT_SCAN_TASK_BATCH_SIZE=512Mi</code>

Key commits (oldest → newest)

See `git log --oneline feature/gpu-native-scan-task ^dev` for the full sequence. High-leverage ones:

- `9a7ee05` — scan task skeleton
- `0c3aeca` — fused bitpacking (−26%)
- `6de87fb` — batched bitpacking (−13%)

- `6bda3b8` — batched two-pass strings
- `f778206` — kernel opts bundle (−18%)
- `685682c` — parallel scan tasks
- `938898f` — lazy mmap (6.9× cold)
- `_(this session)_` — pure-Sirius atomic fast path (−8% warm)

Reference docs (in-repo)

- `docs/super-sirius/benchmarks/2026-04-16_gh200/README.md` — raw CSVs + nsys profiles
- `docs/super-sirius/benchmarks/gh200_sf100_clickbench_20260416.md` — GH200 report
- `docs/scan-optimization-guide.md` — next-step optimization sketches (items 1–4 above)
- `docs/duckdb-pin-overhead-analysis.md` — Pin overhead breakdown
- `active_scan_interleaving.md` (memory) — design for ring buffer work